INTERVIEW QUESTIONS TO LAND AN AI ENGINEER JOB

# RAG Pipeline Components

*Test every layer of the RAG pipeline from ingestion to evaluation.*

**Q01** **How do you evaluate retrieval quality separately from generation quality?**

A INTERVIEW-READY ANSWER

This is important because if you only evaluate end-to-end, you cannot tell whether a bad answer was caused by bad retrieval or bad generation. Separating them helps you fix the right thing.

## EVALUATING RETRIEVAL INDEPENDENTLY

| POINT | WHAT IT MEANS |
|---|---|
| Build a labeled evaluation set | 100 to 200 questions where you know which chunks should be retrieved |
| Measure Recall@K | of the top K retrieved chunks, how many are in the expected set? |
| Measure MRR (Mean Reciprocal Rank) | is the best chunk ranked near the top? |
| Measure NDCG | a ranking-aware metric that rewards placing the most relevant chunk at position 1 |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q01** How do you evaluate retrieval quality separately from generation quality?

A INTERVIEW-READY ANSWER · CONTINUED

## EVALUATING GENERATION INDEPENDENTLY

Given perfect context (manually curated), does the LLM produce a correct answer?

### THIS ISOLATES GENERATION QUALITY FROM RETRIEVAL NOISE

If generation is fine with perfect context but fails with retrieved context, retrieval is your bottleneck

### END-TO-END EVALUATION

### MEASURE ANSWER CORRECTNESS AGAINST GROUND TRUTH ANSWERS

Tools like RAGAS compute faithfulness and answer relevance automatically

> Use human evaluators for a sample of questions monthly, especially for high-stakes domains

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q02** **How do embedding dimensions and model choice impact retrieval quality and performance?**

A INTERVIEW-READY ANSWER

Embedding dimensions affect storage, compute cost, and index performance, but dimension size alone does not guarantee better semantic quality.

KEY POINTS

1. Model architecture matters more than dimension size Two models with the same dimension (e.g., 768) can perform very differently depending on training data and architecture.

2. HIGHER DIMENSIONS INCREASE STORAGE AND INDEX SIZE FOR EXAMPLE

768 dimensions → smaller index size

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q02** **How do embedding dimensions and model choice impact retrieval quality and performance?**

A INTERVIEW-READY ANSWER · CONTINUED

1536+ DIMENSIONS → HIGHER MEMORY USAGE AND SLOWER ANN SEARCH

3. ANN search performance Vector databases rely on Approximate Nearest Neighbor (ANN) algorithms such as HNSW or IVF. Larger vectors increase memory usage and slightly slow similarity search.

4. Trade-off in practice

TYPICAL PRODUCTION RANGES

384–768 dimensions → efficient for large-scale retrieval

768–1536 dimensions → common balance of quality and cost

>3000 DIMENSIONS → HIGHER COST WITH DIMINISHING RETURNS

The embedding model quality and training data usually influence retrieval quality far more than vector dimension size.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q03** **How would you design session-level memory so a RAG assistant remembers user context during a conversation but prevents data leakage across users?**

A INTERVIEW-READY ANSWER

Session memory must support context persistence within a conversation while ensuring strict isolation between users.

- Session memory structure

EXAMPLE SESSION DATA

- session_id
- user_id
- conversation turns
- summaries
- extracted facts
- user preferences.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q03** How would you design session-level memory so a RAG assistant remembers user context during a conversation but prevents data leakage across users?

A INTERVIEW-READY ANSWER · CONTINUED

Secure storage design

STORE SESSION DATA USING A USER-SCOPED KEY

key = f"session:{user_id}:{session_id}"

This ensures sessions cannot be accessed across users.

Verification layer

EVEN AFTER LOADING THE SESSION, VERIFY THAT

session.user_id == requested_user_id

This prevents accidental cross-session access.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q03** How would you design session-level memory so a RAG assistant remembers user context during a conversation but prevents data leakage across users?

A INTERVIEW-READY ANSWER · CONTINUED

Prompt construction

WHEN BUILDING PROMPTS, ONLY INCLUDE CONTEXT FROM THE CURRENT VERIFIED SESSION

recent conversation

extracted session facts

current query.

AUTOMATIC SESSION EXPIRATION

Sessions typically expire after a fixed inactivity period (e.g., 8 hours) using Redis TTL. After expiry, the user starts with a fresh session.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q04** How would you scale a vector database to handle millions of embeddings?

A INTERVIEW-READY ANSWER

Scaling strategy depends on whether the bottleneck is storage, query throughput, or both.

### SHARDING

Partition the vector index across multiple nodes where each node holds a subset of vectors

### QUERY ALL SHARDS IN PARALLEL AND MERGE TOP-K RESULTS

Most managed solutions like Pinecone, Weaviate, and Qdrant handle this automatically

### QUANTIZATION

Reduce vector size using Product Quantization or Scalar Quantization

↔ Example: compress 1536-d float32 vectors at 6KB each down to about 192 bytes, which is a 30x storage reduction

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q04** How would you scale a vector database to handle millions of embeddings?

A INTERVIEW-READY ANSWER · CONTINUED

There is a slight accuracy trade-off but it is often worth it at scale

TIERED STORAGE

KEEP HOT OR RECENT VECTORS IN MEMORY WITH FAST HNSW

Move older or less-accessed vectors to disk-based indexes like DiskANN

Query routing logic decides which tier to hit

FILTERING PRE-PARTITION

Apply metadata filters before vector search to reduce the search space

Example: filter to "documents from 2024" first, then run ANN on that subset

For most production systems hitting 10M or more vectors, I would use a managed vector DB like Pinecone or Weaviate Cloud that handles sharding and replication out of the box.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q05** **What are embeddings, and how do they enable semantic retrieval?**

A INTERVIEW-READY ANSWER

An embedding is a vector, a list of numbers, that represents the meaning of a piece of text. The key idea is that texts with similar meanings end up close together in vector space.

For example, "How do I reset my password?" and "Steps to change my login credentials" would have very similar vectors even though they share no words. That is semantic retrieval, finding meaning rather than matching keywords.

IN A RAG SYSTEM

At indexing time, every chunk is converted to a vector and stored

| POINT | WHAT IT MEANS |
| --- | --- |
| At query time, the question is also converted to a vector | We find the chunks whose vectors are closest to the query vector using cosine similarity or dot product |
| Those chunks are passed to the LLM as context | This is fundamentally more powerful than keyword search for natural language questions. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q06** **How would you design a memory layer that stores previously retrieved context so the system avoids repeated vector searches during follow-up questions?**

A INTERVIEW-READY ANSWER

Follow-up questions in a conversation often refer to the same topic, meaning the same documents may be retrieved repeatedly.

A session-level retrieval memory can store previously retrieved context.

Session retrieval memory design

STORE FOR EACH QUERY

- query text
- query embedding
- retrieved documents
- timestamp.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q06** How would you design a memory layer that stores previously retrieved context so the system avoids repeated vector searches during follow-up questions?

A INTERVIEW-READY ANSWER · CONTINUED

EXAMPLE CONCEPT

CLASS SESSIONRETRIEVALMEMORY

ASYNC DEF STORE_RETRIEVED_CONTEXT(SELF, QUERY, QUERY_EMBEDDING, DOCUMENTS)

```
entry = {
    "query": query,
    "embedding": query_embedding,
    "documents": documents
}
await redis.set(key, entry)
```

Context reuse logic

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q06** How would you design a memory layer that stores previously retrieved context so the system avoids repeated vector searches during follow-up questions?

A INTERVIEW-READY ANSWER · CONTINUED

## WHEN A NEW QUERY ARRIVES

- Generate query embedding
- Search session memory for similar previous queries
- If similarity is high, reuse the stored documents.
- If similarity is low, perform a fresh retrieval.
- Merging results

## FOR FOLLOW-UPS, COMBINE

- reused context
- newly retrieved documents
- while removing duplicates.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q07** **What if retrieval fails due to wording mismatch?**

A INTERVIEW-READY ANSWER

Wording mismatch is when the user's phrasing and the document's phrasing describe the same concept but use different words. The embedding model may not bridge the gap fully.

Example: User asks "how do I cancel my subscription?" but the document says "steps to terminate your account membership." A weak embedding model treats these as different.

SOLUTIONS

| POINT | WHAT IT MEANS |
| --- | --- |
| Query expansion | Generate multiple paraphrases of the query and retrieve for each. "Cancel subscription," "terminate account," "end membership," and "stop billing" all retrieve candidates. Merge and deduplicate. This dramatically improves recall for wording-sensitive queries. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

## Q07 What if retrieval fails due to wording mismatch?

A INTERVIEW-READY ANSWER · CONTINUED

| POINT | WHAT IT MEANS |
| --- | --- |
| HyDE (Hypothetical Document Embedding) | Ask the LLM to generate a short hypothetical answer to the query. That hypothetical answer uses vocabulary closer to how documents are written, so it retrieves better than the original short query. |
| BM25 as a fallback | If dense retrieval returns nothing above the similarity threshold, fall back to BM25 keyword search. At least it finds exact term matches. |
| Synonym expansion | For domain-specific systems, maintain a controlled vocabulary or thesaurus. Expand queries with known synonyms before retrieval. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q08** **What is a vector database, and why is ANN search required instead of exact search?**

A INTERVIEW-READY ANSWER

A vector database is a storage system optimized for storing and searching high-dimensional vectors. Instead of filtering rows by column values like a traditional database, it finds vectors that are most similar to a query vector.

Why not exact search? Exact search computes the distance between the query vector and every single stored vector. For 1 million vectors of 1536 dimensions, that is billions of floating-point operations per query, which is completely impractical at scale.

ANN (Approximate Nearest Neighbor) search trades a tiny bit of accuracy for massive speed gains. Instead of checking every vector, it uses smart index structures to check only a small fraction and still returns results that are 95 to 99% as accurate as exact search.

> **In practice, for RAG systems, that small accuracy trade-off is irrelevant. What matters is returning relevant chunks in under 100ms, and ANN makes that possible.**

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q09** **How do indexing techniques like HNSW improve vector search performance in ANN systems?**

A INTERVIEW-READY ANSWER

HNSW (Hierarchical Navigable Small World) is one of the most widely used indexing methods for Approximate Nearest Neighbor (ANN) search. It organizes vectors into a multi-layer graph structure designed to enable efficient navigation through high-dimensional space.

CORE IDEA

- The index consists of multiple graph layers.
- The top layers contain fewer nodes with long-range connections that allow fast global navigation.
- The lower layers contain more nodes with dense local connections for fine-grained search.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q09** How do indexing techniques like HNSW improve vector search performance in ANN systems?

A INTERVIEW-READY ANSWER · CONTINUED

## SEARCH PROCESS

- The search begins at the top layer using a greedy search to move toward nodes closest to the query vector.
- Once a good candidate region is found, the algorithm moves down to the next layer.
- This process continues until the bottom layer, where the nearest neighbors are refined and returned.

## CITY NAVIGATION ANALOGY

- Top layer → highways to reach the correct district quickly
- Middle layers → local roads to reach the right street

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q09** How do indexing techniques like HNSW improve vector search performance in ANN systems?

A INTERVIEW-READY ANSWER · CONTINUED

Bottom layer → the exact house

## PERFORMANCE CHARACTERISTICS

HNSW dramatically reduces the number of distance computations compared to brute-force search.

Average search complexity behaves roughly logarithmically in practice, but it is not guaranteed O(log n) because performance depends on graph structure and parameters.

It typically achieves high recall (95-99%) with millisecond-level latency even on millions of vectors.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q09** How do indexing techniques like HNSW improve vector search performance in ANN systems?

A INTERVIEW-READY ANSWER · CONTINUED

## KEY TUNING PARAMETERS

| POINT | WHAT IT MEANS |
|---|---|
| M → number of graph connections per node (controls graph density) | ef_construction → controls index construction quality vs build time |
| ef_search → controls recall vs query speed | Higher ef_search increases accuracy but also increases latency. |

## TRADE-OFFS

HNSW is memory-intensive because the entire graph structure is usually kept in RAM.

For extremely large datasets (hundreds of millions or billions of vectors), memory cost becomes a major design constraint.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q10** **Compare bi-encoder and cross-encoder models in the context of re-ranking.**

A INTERVIEW-READY ANSWER

### BI-ENCODER

#### ENCODES THE QUERY AND DOCUMENT INDEPENDENTLY INTO SEPARATE VECTORS

Similarity is computed as dot product or cosine between the two vectors

Very fast because document vectors can be pre-computed and stored

#### USED FOR FIRST-STAGE RETRIEVAL AT SCALE

Limitation: the query and document never see each other during encoding, so nuanced relevance is lost

### CROSS-ENCODER

↔ Takes the query and document together as a single input

Attends to both simultaneously using full self-attention

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q10** **Compare bi-encoder and cross-encoder models in the context of re-ranking.**

A INTERVIEW-READY ANSWER · CONTINUED

### PRODUCES A SINGLE RELEVANCE SCORE, NOT A VECTOR

Much more accurate because it models the interaction between query and document directly

Too slow for first-stage retrieval (no pre-computation possible) but perfect for re-ranking a small shortlist

### HOW THEY WORK TOGETHER

Bi-encoder retrieves top 20 to 50 candidates quickly

Cross-encoder re-ranks those 20 to 50 with high accuracy

### FINAL TOP 5 GO TO THE LLM

This two-stage setup gives you the speed of bi-encoders and the precision of cross-encoders.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q11** **How do metadata filtering and hybrid conditions improve retrieval?**

A INTERVIEW-READY ANSWER

Pure vector search finds semantically similar content, but sometimes you need more precision.

| POINT | WHAT IT MEANS |
| --- | --- |
| Metadata filtering example | User asks: "What is the refund policy for enterprise customers?" |
| Without filtering | might retrieve refund policies for all customer tiers |
| With filtering | customer_tier = "enterprise" narrows the search space first, then runs vector search on just enterprise docs |

This dramatically improves precision without sacrificing recall.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q11** How do metadata filtering and hybrid conditions improve retrieval?

A INTERVIEW-READY ANSWER · CONTINUED

## HYBRID SEARCH COMBINING VECTOR AND KEYWORD

Combine dense vector search for semantics with BM25 keyword search

## USE RECIPROCAL RANK FUSION TO MERGE THE TWO RESULT LISTS

This handles cases where exact terms matter, like product codes, names, or legal clause numbers, that pure vector search might miss

Example: Query "CVE-2024-1234 vulnerability fix." Vector search finds semantically related security content. BM25 finds the exact CVE number. Hybrid gives you both.

Most production RAG systems use hybrid retrieval because it handles a much wider range of query types robustly.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q12** **What is a chunk in a RAG system, and why is chunking necessary?**

A INTERVIEW-READY ANSWER

A chunk is a small, self-contained piece of a document, typically a few sentences or a paragraph. Instead of storing entire documents, we break them into chunks and embed each one separately.

## WHY IT IS NECESSARY

- Embedding models have token limits. You cannot embed a 200-page document as one unit.
- Precision. Retrieving a specific paragraph is far more useful than retrieving an entire document.
- LLM context limits. Retrieved content has to fit in the LLM's context window. Smaller chunks give you more control.
- Relevance. A 10-page document might have only one relevant paragraph. Chunking lets you surface exactly that paragraph.
- Think of it like indexing a textbook. You do not retrieve the whole book, you retrieve the exact page.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q13** **What are guardrails in a RAG system, and why are they necessary?**

A INTERVIEW-READY ANSWER

Guardrails are safety and quality controls placed around the inputs and outputs of a RAG system. They enforce boundaries on what the system can accept, process, and return.

Without guardrails, a RAG system can be manipulated to produce harmful outputs, leak sensitive information, answer out-of-scope questions confidently, or be hijacked through prompt injection.

TYPES OF GUARDRAILS

| POINT | WHAT IT MEANS |
|---|---|
| Input guardrails | Validate and filter user queries before they hit the retriever. Block offensive, harmful, or clearly out-of-scope queries. |
| Retrieval guardrails | Ensure only authorized documents are retrieved for a given user or role. Enforce access control at the chunk level. |
| Output guardrails | Validate the generated answer before returning it. Check for toxic content, sensitive data exposure, or low-faithfulness answers. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q13** **What are guardrails in a RAG system, and why are they necessary?**

A INTERVIEW-READY ANSWER · CONTINUED

| POINT | WHAT IT MEANS |
| --- | --- |
| Domain guardrails | Restrict the system to only answer questions within a defined scope. A customer support bot should not answer questions about unrelated topics. |

### WHY NECESSARY

### LLMS ARE POWERFUL BUT EASILY MANIPULATED

Users in enterprise systems may not have permission to access all data

Regulatory compliance in industries like healthcare and finance requires strict output controls

A single bad output can cause significant reputational or legal damage

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

## Q14 How do you rewrite user queries?

A INTERVIEW-READY ANSWER

Query rewriting transforms a raw user query into a cleaner, more retrieval-friendly version without changing its intent.

### TECHNIQUES I USE

| POINT | WHAT IT MEANS |
|---|---|
| Contextualization | Add relevant context from the conversation history to make the query self-contained. Anaphoric queries like "what about the second option?" become "what are the pricing details for the Standard plan?" |
| Expansion | Add related terms or rephrasings to broaden coverage. One query becomes 3 to 5 variants. |
| Decomposition | Break compound questions into sub-questions. "What is the pricing and cancellation policy?" becomes two separate retrieval queries for more precise results. |
| Formalization | Convert informal or colloquial queries into the formal language used in documents. "How do I get my money back?" becomes "What is the refund process?" |
| Specificity enhancement | Make vague queries more specific using context clues. "Tell me about the update" becomes "Tell me about the March 2024 product update." |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q15** **If your knowledge base updates frequently, how would you invalidate cached embeddings or retrieval results to ensure users receive the latest information?**

A INTERVIEW-READY ANSWER

When documents update frequently, cache invalidation must be fast and precise.

### EVENT-DRIVEN INVALIDATION

Document updates trigger an event that removes all cache entries referencing that document.

### EXAMPLE CONCEPT

ASYNC DEF HANDLE_DOCUMENT_UPDATED(DOCUMENT_ID)

```
tag_key = f"tag:{document_id}"
dependent_keys = await redis.smembers(tag_key)
```

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q15** If your knowledge base updates frequently, how would you invalidate cached embeddings or retrieval results to ensure users receive the latest information?

A INTERVIEW-READY ANSWER · CONTINUED

### IF DEPENDENT_KEYS

```
await redis.delete(*dependent_keys)
await redis.delete(tag_key)
await enqueue_reingestion(document_id)
```

Re-embedding pipeline

### AFTER INVALIDATION

Re-embed the updated document

Update the vector index

### TTL SAFETY FALLBACK

Even with event-driven invalidation, all cache entries should have a TTL to prevent stale content if an update event is missed.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q16** **What techniques can reduce irrelevant or noisy retrieved context?**

A INTERVIEW-READY ANSWER

Noisy context is one of the biggest reasons RAG systems give bad answers. The LLM tries to use everything you give it, even if it should not.

## TECHNIQUES I USE

| POINT | WHAT IT MEANS |
| --- | --- |
| **Similarity score filtering** | Drop any chunk below a minimum cosine similarity threshold before passing to the LLM. If nothing passes the threshold, return "I don't have information on this" rather than hallucinating. |
| **Re-ranking** | Use a cross-encoder to re-score retrieved chunks for relevance to the specific query. This removes false positives that looked similar at the vector level but are not actually relevant. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q16** **What techniques can reduce irrelevant or noisy retrieved context?**

A INTERVIEW-READY ANSWER · CONTINUED

| POINT | WHAT IT MEANS |
| --- | --- |
| Metadata filtering | Narrow the search space before vector search. For example, only search within a specific product category or date range. Fewer irrelevant documents means fewer irrelevant chunks. |
| Maximal Marginal Relevance (MMR) | When multiple chunks are near-identical, MMR diversifies the selection so you get broad coverage instead of five versions of the same paragraph. |
| Contextual compression | After retrieval, use an LLM to extract only the sentences from each chunk that are actually relevant to the query. You pass compressed, focused context instead of whole chunks. |

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q17** **How do ambiguous or underspecified queries affect retrieval, and how can you handle them?**

A INTERVIEW-READY ANSWER

Ambiguous queries are a huge real-world problem. When a user types "what is the limit?", the retriever has no idea if they mean API rate limit, credit limit, file size limit, or something else entirely. The vector for that query is vague, and retrieval pulls in chunks from completely different topics.

### HOW THIS HURTS RETRIEVAL

The query embedding is a broad, unfocused vector

### TOP RETRIEVED CHUNKS COME FROM MULTIPLE UNRELATED TOPICS

The LLM either picks one interpretation arbitrarily or gives a confused answer

### HOW I HANDLE IT

Query rewriting using an LLM. Before retrieval, pass the query through a rewriting prompt: "Rewrite this query to be specific and self-contained based on the conversation history." Short queries become richer and more targeted.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q17** **How do ambiguous or underspecified queries affect retrieval, and how can you handle them?**

A INTERVIEW-READY ANSWER · CONTINUED

Query expansion. Generate 2 to 3 alternative phrasings of the query and retrieve for each. Merge and deduplicate the results. This increases the chance of hitting the right chunk.

Hypothetical Document Embedding (HyDE). Ask the LLM to generate a hypothetical answer to the query, then embed that answer and use it as the retrieval query. Hypothetical answers are more detailed than short queries and retrieve better.

Clarification fallback. If the top retrieved chunks cover very different topics with similar scores, the system can ask the user a clarifying question before generating an answer.

Session context. Use conversation history to add context to the current query. "What is the limit?" in a conversation about API usage becomes much easier to retrieve for.

7 Hybrid Search and Re-Ranking

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q18** **What is data ingestion in a RAG system, and how would you design a scalable ingestion pipeline for millions of documents?**

A INTERVIEW-READY ANSWER

Data ingestion is basically the first step, collecting raw documents from different sources and getting them into the system so they can be searched and used by the LLM.

FOR A SCALABLE PIPELINE, HERE'S HOW I'D APPROACH IT

- Source connectors pull documents from S3, SharePoint, databases, web crawlers, etc.
- Message queue (Kafka / SQS) buffers everything so the system handles traffic spikes without crashing. Don't process directly from the source.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

**Q18** What is data ingestion in a RAG system, and how would you design a scalable ingestion pipeline for millions of documents?

A INTERVIEW-READY ANSWER · CONTINUED

Parallel workers pick from the queue, parse, chunk, embed, and store. Scale horizontally by adding more workers.

Idempotency means hashing every document before processing. If the hash already exists, skip it. This prevents reprocessing on reruns.

Dead-letter queue captures failed documents separately so one bad file doesn't block everything.

Simple flow: Source > Queue > Workers > Vector Store

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

# This is only a selected preview.

Get the complete RAG Interview Kit for deeper preparation across the full curriculum.

- Full question bank
- Detailed answers
- Follow-up questions
- Real-world scenarios

↓ Check the caption to enroll